

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

THEJESH VEPATI BALAJI(1BF24CS319)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Thejesh Vepati Balaji (1BF24CS319) who is bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2026. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Ambuja K
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)	3
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	20
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	23
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	32
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	38
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	46
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	50

1. Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) FCFS

```
#include <stdio.h>
```

```
struct Process { int pid; int at; int bt; int ct; int wt; int tat; };
```

```
int main() { int n, i, j; printf("Enter number of processes: "); scanf("%d", &n);
```

```
    struct Process p[n], temp;

    for(i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("\nProcess %d\n", i + 1);
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
    }

    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(p[i].at > p[j].at) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    int current_time = 0;

    for(i = 0; i < n; i++) {
        if(current_time < p[i].at)
            current_time = p[i].at;

        p[i].ct = current_time + p[i].bt;
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;

        current_time = p[i].ct;
    }

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

    float total_wt = 0, total_tat = 0;

    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt,
            p[i].ct, p[i].tat, p[i].wt);
    }
}
```

```

        total_wt += p[i].wt;
        total_tat += p[i].tat;
    }

    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
    printf("Thejesh Vepati Balaji 1BF24CS319");

    return 0;
}

```

OUTPUT:

The screenshot shows a code editor window titled "FCFS.c - Codes::Blocks 25.03" with a menu bar (File, Edit, View, Search, Project, Build, Debug, Fortran, wxSmith, Tools, Tools+, Plugins, DoxyBlocks, Settings, Help) and a toolbar. The code editor displays the following C code:

```

38 }
39
40 int current_time = 0;
41

```

Below the code editor is a terminal window titled "C:\Users\BMSCECSE-L4\Desktop" showing the execution of the program. The terminal output is as follows:

```

Enter number of processes: 4

Process 1
Arrival Time: 0
Burst Time: 2

Process 2
Arrival Time: 1
Burst Time: 2

Process 3
Arrival Time: 5
Burst Time: 3

Process 4
Arrival Time: 6
Burst Time: 4

PID  AT  BT  CT  TAT  WT
P1   0   2   2   2   0
P2   1   2   4   3   1
P3   5   3   8   3   0
P4   6   4  12   6   2

Average Waiting Time = 0.75
Average Turnaround Time = 3.50
Thejesh Vepati Balaji 1BF24CS319
Process returned 0 (0x0)   execution time : 32.240 s
Press any key to continue.

```

At the bottom of the terminal window, there is a status bar showing build information:

```

=== Build file: "no target" in "no project" (compiler: unknown) ===
=== Build finished: 0 error(s), 0 warning(s), (0 minute(s), 0 second(s)) ===

```

b) SJF (pre-emptive)

```
struct Process p[n];
for (int i = 0; i < n; i++) {
    printf("Enter Arrival Time and Burst Time for Process %d: ", i+1);
    p[i].pid = i+1;
    scanf("%d %d", &p[i].arrival, &p[i].burst);
    p[i].remaining = p[i].burst;
    p[i].finished = false;
}

int current_time = 0;
int completed = 0;

while (completed < n) {
    int idx = -1;
    int min_remaining = 1e9;

    for (int i = 0; i < n; i++) {
        if (!p[i].finished && p[i].arrival <= current_time && p[i].remaining
< min_remaining) {
            min_remaining = p[i].remaining;
            idx = i;
        }
    }

    if (idx == -1) {
        current_time++; // No process ready
    } else {
        p[idx].remaining--;
        current_time++;

        if (p[idx].remaining == 0) {
            p[idx].completion = current_time;
            p[idx].turnaround = p[idx].completion - p[idx].arrival;
            p[idx].waiting = p[idx].turnaround - p[idx].burst;
            p[idx].finished = true;
            completed++;
        }
    }
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].arrival, p[i].burst,
        p[i].completion, p[i].turnaround, p[i].waiting);
}
printf("1BF24CS319");

return 0;
}
```

OUTPUT:

```
Enter number of processes: 5
Enter Arrival Time and Burst Time for Process 1: 0
5
Enter Arrival Time and Burst Time for Process 2: 1
3
Enter Arrival Time and Burst Time for Process 3: 3
4
Enter Arrival Time and Burst Time for Process 4: 6
3
Enter Arrival Time and Burst Time for Process 5: 7
11

Process AT      BT      CT      TAT      WT
P1      0       5       8       8       3
P2      1       3       4       3       0
P3      3       4      15      12       8
P4      6       3      11       5       2
P5      7      11      26      19       8
1BF24CS319
Process returned 0 (0x0)  execution time : 23.582 s
Press any key to continue.
```


b) SJF (non pre-emptive)

```
#include <stdio.h>
```

```
struct Process { int pid; int at; int bt; int ct; int tat; int wt; int rt; int completed; }; int main() { int n; printf("Enter number of processes: "); scanf("%d", &n); struct Process p[n]; for(int i = 0; i < n; i++) { p[i].pid = i + 1; printf("Enter Arrival Time for Process %d: ", i+1); scanf("%d", &p[i].at); printf("Enter Burst Time for Process %d: ", i+1); scanf("%d", &p[i].bt); p[i].completed = 0; } int completed = 0, current_time = 0; float total_wt = 0, total_tat = 0, total_rt = 0; while(completed < n) { int idx = -1; int min_bt = 1e9; for(int i = 0; i < n; i++) { if(p[i].at <= current_time && p[i].completed == 0) { if(p[i].bt < min_bt) { min_bt = p[i].bt; idx = i; } } } if(idx == -1) { current_time++; } else { p[idx].ct = current_time + p[idx].bt; p[idx].tat = p[idx].ct - p[idx].at; p[idx].wt = p[idx].tat - p[idx].bt; p[idx].rt = p[idx].wt; current_time = p[idx].ct; p[idx].completed = 1; completed++;
```

```
        total_wt += p[idx].wt;
        total_tat += p[idx].tat;
        total_rt += p[idx].rt;
    }
}
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for(int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt,
        p[i].ct, p[i].tat, p[i].wt, p[i].rt);
}
printf("\nAverage Waiting Time = %.2f", total_wt/n);
printf("\nAverage Turnaround Time = %.2f", total_tat/n);
printf("\nAverage Response Time = %.2f\n", total_rt/n);
printf("\n 1BF24CS319");

return 0;

}
```

OUTPUT:

```
"C:\Users\BMSCECSE-L4\Desktop" X + v
Enter number of processes: 4
Enter Arrival Time for Process 1: 0
Enter Burst Time for Process 1: 6
Enter Arrival Time for Process 2: 0
Enter Burst Time for Process 2: 8
Enter Arrival Time for Process 3: 0
Enter Burst Time for Process 3: 7
Enter Arrival Time for Process 4: 0
Enter Burst Time for Process 4: 3

PID    AT    BT    CT    TAT    WT    RT
P1      0     6     9     9      3     3
P2      0     8    24    24    16    16
P3      0     7    16    16     9     9
P4      0     3     3     3     0     0

Average Waiting Time = 7.00
Average Turnaround Time = 13.00
Average Response Time = 7.00

1BF24CS319
Process returned 0 (0x0)   execution time : 29.900 s
Press any key to continue.
|
```

c) Priority (pre emptive)

```
#include <stdio.h>
```

```
struct Process { int pid; int at; int bt; int bt_remaining; int priority; int ct; int tat; int wt; };
```

```
int main() { int n; printf("Enter number of processes: "); scanf("%d", &n);
```

```
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter Arrival Time, Burst Time, Priority for P%d: ", i + 1);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].priority);
        p[i].bt_remaining = p[i].bt;
    }
```

```
    int completed = 0, time = 0;
    while (completed != n) {
        int idx = -1;
        int highest_priority = 999999;
        for (int i = 0; i < n; i++) {
            if (p[i].at <= time && p[i].bt_remaining > 0) {
                if (p[i].priority < highest_priority) {
                    highest_priority = p[i].priority;
                    idx = i;
                }
                else if (p[i].priority == highest_priority) {
                    if (p[i].at < p[idx].at) {
                        idx = i;
                    }
                }
            }
        }
    }
```

```
    if (idx != -1) {
        p[idx].bt_remaining--;
        time++;

        if (p[idx].bt_remaining == 0) {
            p[idx].ct = time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;
            completed++;
        }
        else {
            time++;
        }
    }
}
float total_tat = 0 , total_wt = 0;
```

```
for(int i=0 ; i<n ; i++){
```

```
total_tat += p[i].tat;
```

```
total_wt += p[i].wt;
```

```

}

printf("\nProcess\tAT\tBT\tPriority\tCT\tTAT\tWT\n");

for (int i = 0; i < n; i++){

printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt, p[i].priority, p[i].ct, p[i].tat,
p[i].wt); }

printf("\nAverage Waiting Time = %.2f", total_wt/n);

printf("\nAverage Turnaround Time = %.2f\n", total_tat/n);

printf("1BF24CS319"); return 0; }

```

OUTPUT:

```

1
5
Enter Arrival Time, Burst Time, Priority for P5: 5
6
2
Enter Arrival Time, Burst Time, Priority for P6: 6
5
6
Enter Arrival Time, Burst Time, Priority for P7: 10
1
1

```

Process	AT	BT	Priority	CT	TAT	WT
P1	0	8	3	15	15	7
P2	1	2	4	17	16	14
P3	3	4	4	21	18	14
P4	4	1	5	22	18	17
P5	5	6	2	12	7	1
P6	6	5	6	27	21	16
P7	10	1	1	11	1	0

```

Average Waiting Time = 9.86
Average Turnaround Time = 13.71

```

c) Priority (Non pre-emptive)

```
struct Process {  
  
    int pid;  
  
    int bt;  
  
    int art;  
  
    int pri;  
  
    int ct;  
  
    int tat;  
  
    int wt;  
  
    int isCompleted;  
  
};
```

```
int main() {  
  
    int n,  
  
    current_time = 0,  
  
    completed = 0;  
  
    float avgwt = 0,  
  
    avgtat = 0;  
  
    struct Process  
  
    p[100];  
  
    printf("Enter total  
  
number of  
  
processes: ");
```

```

scanf("%d", &n);

for(int i = 0; i < n;
i++) {

    printf("\nEnter
details for Process
%d (AT BT
Priority): ", i + 1);

    p[i].pid = i + 1;

    scanf("%d %d
%d", &p[i].art,
&p[i].bt, &p[i].pri);

p[i].isCompleted =
0;

}

while(completed !
= n) {

    int idx = -1;

    int max_pri =
9999;

    for(int i = 0; i <
n; i++) {

        if(p[i].art <=

```

```

current_time &&

p[i].isCompleted ==

0) {

    if(p[i].pri <

max_pri) {

        max_pri

= p[i].pri;

        idx = i;

    }

    if(p[i].pri

== max_pri) {

        if(idx !=

-1 && p[i].art <

p[idx].art) {

            idx =

i;

        }

    }

}

if(idx != -1) {

    p[idx].ct =

```

```

current_time +

p[idx].bt;

    p[idx].tat =

p[idx].ct - p[idx].art;

    p[idx].wt =

p[idx].tat - p[idx].bt;


p[idx].isCompleted

= 1;

    current_time

= p[idx].ct;

    completed+

+;

    } else {


current_time++;

    }

}


printf("\nPID\tAT\t

BT\tPri\tCT\tTAT\t

WT\n");

for(int i = 0; i < n;

```



```
i++) {  
  
    printf("%d\t%d\t%d\t%d  
\t%d\t%d\t%d\t%d\t%d\t\nd\n", p[i].pid, p[i].art,  
        p[i].bt, p[i].pri,  
        p[i].ct, p[i].tat,  
        p[i].wt);  
  
        avgwt +=  
        p[i].wt;  
  
        avgtat +=  
        p[i].tat;  
  
    }  
  
    printf("\nAverage  
Waiting Time:  
%.2f", avgwt / n);  
  
    printf("\nAverage  
Turnaround Time:  
%.2fn", avgtat / n);  
  
    printf("1BF24CS9")  
  
    ;  
  
    return 0;
```

}

OUTPUT:

Enter total number of processes: 4

Enter details for Process 1 (AT BT Priority): 1

3

2

Enter details for Process 2 (AT BT Priority): 2

4

3

Enter details for Process 3 (AT BT Priority): 3

5

6

Enter details for Process 4 (AT BT Priority): 4

2

5

PID	AT	BT	Pri	CT	TAT	WT
1	1	3	2	4	3	0
2	2	4	3	8	6	2
3	3	5	6	15	12	7
4	4	2	5	10	6	4

Average Waiting Time: 3.25

Average Turnaround Time: 6.75

thejesh@192:~/Documents/OS/Practice\$

d) Round Robin

```
#include <stdio.h> #include <stdbool.h>
```

```
struct Process { int id; int arrival_time; int burst_time; int remaining_time; int waiting_time; int  
turnaround_time; int completion_time; bool is_completed; bool in_queue; };
```

```
int main() { int n, time_quantum, completed = 0, current_time = 0;
```

```
printf("Enter number of processes: ");  
scanf("%d", &n);  
printf("Enter time quantum: ");  
scanf("%d", &time_quantum);  
  
struct Process p[n];  
int queue[1000];  
int front = 0, rear = 0;  
  
for (int i = 0; i < n; i++) {  
    p[i].id = i + 1;  
    printf("Enter Arrival Time and Burst Time for Process %d: ", i + 1);  
    scanf("%d %d", &p[i].arrival_time, &p[i].burst_time);  
    p[i].remaining_time = p[i].burst_time;  
    p[i].is_completed = false;  
    p[i].in_queue = false;  
}  
  
for (int i = 0; i < n; i++) {  
    if (p[i].arrival_time <= current_time) {  
        queue[rear++] = i;  
        p[i].in_queue = true;  
    }  
}  
  
while (completed < n) {  
    if (front == rear) {  
        current_time++;  
        for (int i = 0; i < n; i++) {  
            if (p[i].arrival_time <= current_time && !p[i].is_completed && !  
p[i].in_queue) {  
                queue[rear++] = i;  
                p[i].in_queue = true;  
            }  
        }  
        continue;  
    }  
  
    int i = queue[front++];  
  
    if (p[i].remaining_time > time_quantum) {  
        current_time += time_quantum;  
        p[i].remaining_time -= time_quantum;  
    } else {  
        current_time += p[i].remaining_time;  
        p[i].remaining_time = 0;  
        p[i].is_completed = true;  
        completed++;  
    }  
}
```

```

        p[i].completion_time = current_time;
        p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
        p[i].waiting_time = p[i].turnaround_time - p[i].burst_time;
    }

    for (int j = 0; j < n; j++) {
        if (p[j].arrival_time <= current_time && !p[j].is_completed && !
p[j].in_queue) {
            queue[rear++] = j;
            p[j].in_queue = true;
        }
    }

    if (!p[i].is_completed) {
        queue[rear++] = i;
    }
}

float total_wt = 0, total_tat = 0;
printf("\nID\tAT\tBT\tCT\tWT\tTAT\n");
for (int i = 0; i < n; i++) {
    total_wt += p[i].waiting_time;
    total_tat += p[i].turnaround_time;
    printf("%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].arrival_time,
p[i].burst_time, p[i].completion_time, p[i].waiting_time,
p[i].turnaround_time);
}

printf("\nAverage Waiting Time: %.2f", total_wt / n);
printf("\nAverage Turnaround Time: %.2f", total_tat / n);
printf("\n1BF24CS319\n");

return 0;
}

```

```
Enter number of processes: 5
Enter time quantum: 2
Enter Arrival Time and Burst Time for Process 1: 0
5
Enter Arrival Time and Burst Time for Process 2: 1
3
Enter Arrival Time and Burst Time for Process 3: 2
1
Enter Arrival Time and Burst Time for Process 4: 3
2
Enter Arrival Time and Burst Time for Process 5: 4
3
```

ID	AT	BT	CT	WT	TAT
1	0	5	13	8	13
2	1	3	12	8	11
3	2	1	5	2	3
4	3	2	9	4	6
5	4	3	14	7	10

```
Average Waiting Time: 5.80
Average Turnaround Time: 8.60
1BF24CS319
```

2. Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h> #include <string.h>
```

```
typedef struct { int id; int arrival; int burst; int completion; int turnaround; int waiting; char type[10]; } Process;
```

```
int main() { int n, sq_count = 0, uq_count = 0; int current_time = 0; float total_wt = 0, total_tat = 0;
```

```
printf("Enter number of processes: ");
scanf("%d", &n);
```

```
Process sq[n], uq[n], all_p[n];
```

```
for (int i = 0; i < n; i++) {
    all_p[i].id = i + 1;
    printf("\nProcess %d Details:\n", i + 1);
    printf("Arrival Time: ");
    scanf("%d", &all_p[i].arrival);
    printf("Burst Time: ");
    scanf("%d", &all_p[i].burst);
    printf("Type (System/User): ");
    scanf("%s", all_p[i].type);

    if (strcmp(all_p[i].type, "System") == 0) {
        sq[sq_count++] = all_p[i];
    } else {
        uq[uq_count++] = all_p[i];
    }
}
```

```
int sq_idx = 0, uq_idx = 0;
while (sq_idx < sq_count || uq_idx < uq_count) {
```

```
    while (sq_idx < sq_count) {
        if (current_time < sq[sq_idx].arrival) {
            current_time = sq[sq_idx].arrival;
        }
        current_time += sq[sq_idx].burst;
        sq[sq_idx].completion = current_time;
        for(int i=0; i<n; i++) {
            if(all_p[i].id == sq[sq_idx].id) all_p[i].completion =
current_time;
        }
        sq_idx++;
    }
```

```
    while (sq_idx == sq_count && uq_idx < uq_count) {
        if (current_time < uq[uq_idx].arrival) {
            current_time = uq[uq_idx].arrival;
        }
    }
```

```

        current_time += uq[uq_idx].burst;
        uq[uq_idx].completion = current_time;

        for(int i=0; i<n; i++) {
            if(all_p[i].id == uq[uq_idx].id) all_p[i].completion =
current_time;
        }
        uq_idx++;
    }
}

printf("\nID\tType\tArr\tBurst\tComp\tTAT\tWait\n");
for (int i = 0; i < n; i++) {
    all_p[i].turnaround = all_p[i].completion - all_p[i].arrival;
    all_p[i].waiting = all_p[i].turnaround - all_p[i].burst;

    total_tat += all_p[i].turnaround;
    total_wt += all_p[i].waiting;

    printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
        all_p[i].id, all_p[i].type, all_p[i].arrival,
        all_p[i].burst, all_p[i].completion,
        all_p[i].turnaround, all_p[i].waiting);
}

printf("\nAverage Turnaround Time: %.2f", total_tat / n);
printf("\nAverage Waiting Time: %.2f\n", total_wt / n);
printf("1BF24CS319\n");

return 0;
}

```

OUTPUT:

```
Enter number of processes: 5
```

```
Process 1 Details:
```

```
Arrival Time: 0
```

```
Burst Time: 4
```

```
Type (System/User): 0
```

```
Process 2 Details:
```

```
Arrival Time: 3
```

```
Burst Time: 2
```

```
Type (System/User): 0
```

```
Process 3 Details:
```

```
Arrival Time: 0
```

```
Burst Time: 5
```

```
Type (System/User): 1
```

```
Process 4 Details:
```

```
Arrival Time: 3
```

```
Burst Time: 2
```

```
Type (System/User): 0
```

```
Process 5 Details:
```

```
Arrival Time: 4
```

```
Burst Time: 3
```

```
Type (System/User): 1
```

Process 4 Details:

Arrival Time: 3

Burst Time: 2

Type (System/User): 0

Process 5 Details:

Arrival Time: 4

Burst Time: 3

Type (System/User): 1

ID	Type	Arr	Burst	Comp	TAT	Wait
1	0	0	4	4	4	0
2	0	3	2	6	3	1
3	1	0	5	11	11	6
4	0	3	2	13	10	8
5	1	4	3	16	12	9

Average Turnaround Time: 8.00

Average Waiting Time: 4.80

1BF24CS319

3. Write a C program to simulate Real-Time CPU Scheduling algorithms:

a) Rate- Monotonic

```
#include <stdio.h>
int find_lcm(int a, int b) {
    int max = (a > b) ? a : b;
    while (1) {
        if (max % a == 0 && max % b == 0) return max;
        max++;
    }
}
int main() {
    int n, i, j, time;
    int burst[10], period[10], remaining[10], last_release[10];

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("Enter the CPU burst times:\n");
    for (i = 0; i < n; i++) scanf("%d", &burst[i]);

    printf("Enter the time periods:\n");
    for (i = 0; i < n; i++) scanf("%d", &period[i]);

    int lcm = period[0];
    for (i = 1; i < n; i++) lcm = find_lcm(lcm, period[i]);
    printf("LCM = %d\n", lcm);

    printf("\nRate Monotone Scheduling:\nPID\tBurst\tPeriod\n");
    float utilization = 0;
    for (i = 0; i < n; i++) {
        printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);
        utilization += (float)burst[i] / period[i];
        remaining[i] = 0;
        last_release[i] = -1;
    }
    printf("%f <= 0.828427 => true\n", utilization);
    printf("Scheduling occurs for %d ms\n\n", lcm);

    int current_task = -1;
    for (time = 0; time < lcm; time++) {
```

```

for (i = 0; i < n; i++) {
    if (time % period[i] == 0) {
        remaining[i] = burst[i];
    }
}
int selected = -1;
int min_period = 10000;
for (i = 0; i < n; i++) {
    if (remaining[i] > 0 && period[i] < min_period) {
        min_period = period[i];
        selected = i;
    }
}
if (selected != current_task) {
    if (selected != -1)
        printf("%dms onwards: Process %d running\n", time, selected + 1);
    else
        printf("%dms onwards: CPU is idle\n", time);
    current_task = selected;
}

if (selected != -1) remaining[selected]--;
}
printf("1BF24CS319 \n");
return 0;
}

```

OUTPUT:

```
Enter the number of processes: 2
Enter the CPU burst times:
20 35
Enter the time periods:
50
100
LCM = 100
```

```
Rate Monotone Scheduling:
PID      Burst   Period
1         20      50
2         35     100
0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms
```

```
0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle
1BF24CS319
```

b) Earliest-deadline

OUTPUT:

```
Enter the number of processes: 3
PID 1 - Burst, Deadline, Period: 2 4 5
PID 2 - Burst, Deadline, Period: 3 7 20
PID 3 - Burst, Deadline, Period: 2 8 10
```

Scheduling occurs for 20 ms

```
0ms: Task 1 is running.
1ms: Task 1 is running.
2ms: Task 2 is running.
3ms: Task 2 is running.
4ms: Task 2 is running.
5ms: Task 3 is running.
6ms: Task 3 is running.
7ms: Task 1 is running.
8ms: Task 1 is running.
9ms: CPU is idle.
10ms: Task 1 is running.
11ms: Task 1 is running.
12ms: Task 3 is running.
13ms: Task 3 is running.
14ms: CPU is idle.
15ms: Task 1 is running.
16ms: Task 1 is running.
17ms: CPU is idle.
18ms: CPU is idle.
19ms: CPU is idle.
1BF24CS319
```

c)Proportional-sharing

```
#include<stdio.h>

#include <stdlib.h>

#include <time.h>

typedef struct

{

char name[5];

int tickets;

}

Process;

int main()

{ int n, total_tickets = 0;

float total_T =0.0;

printf("Enter the number of Processes: ");

scanf("%d", &n); Process p[n];

srand(time(NULL));

for (int i = 0; i < n; i++) {

printf("\nProcess %d:\n", i + 1);

sprintf(p[i].name, "P%d", i + 1);

printf("Tickets: ");

scanf("%d", &p[i].tickets);

total_tickets += p[i].tickets; total_T +=p[i].tickets;

}

printf("\n--- Proportional Share Scheduling ---\n");
```

```

printf("Enter the Time Period for scheduling: "); int m;

scanf("%d",&m);

for (int i = 0; i < m; i++) {

int winning_ticket = rand() % total_tickets + 1;

int accumulated_tickets = 0;

int winner_index;

for (int j = 0; j < n; j++) {

accumulated_tickets += p[j].tickets;

if (winning_ticket <= accumulated_tickets) {

winner_index = j; break;

}

}

printf("Tickets picked: %d, Winner: %s\n", winning_ticket, p[winner_index].name); }

for (int i = 0; i < n; i++) {

printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name, ((p[i].tickets /
total_T) * 100)); printf("1BF24CS319");

}

return 0;

}

```

OUTPUT:

```
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 59, Winner: P3
Tickets picked: 58, Winner: P3
Tickets picked: 34, Winner: P3
Tickets picked: 5, Winner: P1
Tickets picked: 50, Winner: P3

The Process: P1 gets 16.67% of Processor Time.
1BF24CS319
The Process: P2 gets 33.33% of Processor Time.
1BF24CS319
The Process: P3 gets 50.00% of Processor Time.
1BF24CS319
Process returned 0 (0x0)    execution time : 9.405 s
Press any key to continue.
```

ogs

4. Write a C program to simulate:

a) Producer-Consumer problem using
semaphores.

```
#include <stdlib.h>

#define BUFFER_SIZE 2

int mutex = 1;

int full = 0;

int empty = BUFFER_SIZE;

int buffer[BUFFER_SIZE];

int in = 0, out = 0;

int wait(int s) {

    return --s;

}

int signal(int s) {

    return ++s;

}

void producer() {

    if ((mutex == 1)&&(empty != 0)) {

        mutex=wait(mutex);

        empty=wait(empty);

        int item=rand() % 100;

        buffer[in]=item;
```

```

    printf("Produced: %d\n",item);

    in =(in + 1)%BUFFER_SIZE;

    mutex=signal(mutex);

    full=signal(full);

} else {

    printf("Buffer Overflow\n");

}

}

void consumer() {

    if ((mutex == 1) && (full != 0)) {

        mutex = wait(mutex);

        full = wait(full);

        int item = buffer[out];

        printf("Consumed: %d\n",item);

        out=(out + 1)%BUFFER_SIZE;

        mutex=signal(mutex);

        empty=signal(empty);

    } else {

        printf("Buffer Underflow\n");

    }

}

int main() {

    int choice;

```

```

while (1) {

    printf("\n1. Produce\n2. Consume\n3.
Exit\n");

    printf("Enter your choice: ");

    scanf("%d",&choice);

    switch (choice) {

        case 1:

            producer();

            break;

        case 2:

            consumer();

            break;

        case 3:

            exit(0);

        default:

            printf("Invalid choice!\n");

    }

}

```

\OUTPUT:

```
1. Produce
2. Consume
3. Exit
Enter your choice: 1
Produced: 41

1. Produce
2. Consume
3. Exit
Enter your choice: 1
Produced: 67

1. Produce
2. Consume
3. Exit
Enter your choice: 1
Buffer Overflow

1. Produce
2. Consume
3. Exit
Enter your choice: 2
Consumed: 41

1. Produce
2. Consume
3. Exit
Enter your choice: 2
Consumed: 67

1. Produce
2. Consume
3. Exit
Enter your choice: 2
Buffer Underflow

1. Produce
2. Consume
3. Exit
Enter your choice:
```

b) Dining-Philosopher's

problem #include <stdio.h>

#define MAX 10

int n;

int hungry;

int pos[MAX];

int canEat(int p, int eating[])

{ int left = (p + n - 1) % n;

int right = (p + 1) % n;

if (eating[left] == 0 && eating[right] == 0)

return 1;

return 0;

}

void oneAtATime() {

int eating[MAX] = {0};

printf("Allow one philosopher to eat at any time\n");

for (int i = 0; i < hungry; i++) {

int p = pos[i];

for (int j = 0; j < hungry; j++)

{ printf("P %d is waiting\n",

pos[j]);

}

if (canEat(p, eating))

{ eating[p] = 1;

printf("P %d is granted to eat\n", p);

printf("P %d has finished eating\n", p);

eating[p] = 0;

}

}

}

void twoAtATime() {

int eating[MAX] = {0};

printf("Allow two philosophers to eat at any time\n");

```

for (int i = 0; i < hungry; i++)
    { int p = pos[i];

    if (canEat(p, eating))
        { eating[p] = 1;
          printf("P %d is granted to eat\n", p);
        } else {
          printf("P %d is waiting\n", p);
        }
    }

for (int i = 0; i < hungry; i++)
    { int p = pos[i];
      if (eating[p]) {
        printf("P %d has finished eating\n", p);
        eating[p] = 0;
      }
    }
}

int main()
{ int
  choice;

  printf("Enter the total number of philosophers:
  "); scanf("%d", &n);

  printf("How many are hungry: ");
  scanf("%d", &hungry);

  for (int i = 0; i < hungry; i++) {
    printf("Enter philosopher %d position (1 to %d): ", i + 1, n);
    scanf("%d", &pos[i]);
    pos[i]--;
  }

  while (1) {
    printf("\n1. One can eat at a time\n");
    printf("2. Two can eat at a time\n");
    printf("3. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
        { case 1:

```

```

        oneAtATime();
        break;
    case 2:
        twoAtATime();
        break;
    case 3:
        return
    0; default:
        printf("Invalid choice\n");
    }
}

return 0;
}

```

OUTPUT:

```

thejesh@192:~/Documents/DiningPhilosphers$ /home/thejesh/Documents/DiningPhilosphers/Dining
Enter number of philosophers: 5
--- Resource Allocation Protocol Sequences
Philosopher 1 allocates left chopstick 0 and right chopstick 1 -> Status: EATIN
Philosopher 2 allocates left chopstick 1 and right chopstick 2 -> Status: EATIN
Philosopher 3 allocates left chopstick 2 and right chopstick 3 -> Status: EATIN
Philosopher 4 allocates left chopstick 3 and right chopstick 4 -> Status: EATIN
Philosopher 5 allocates left chopstick 4 and right chopstick 0 -> Status: EATIN
thejesh@192:~/Documents/DiningPhilosphers$

```

5. Write a C program to simulate:

a) Bankers' algorithm for the purpose of deadlock

avoidance. #include <stdio.h>

```
int main()
{
    int n, m;

    printf("Enter number of processes -- ");
    scanf("%d", &n);

    printf("Enter number of resources -- ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];
    int finish[n], safe[n];

    for (int i = 0; i < n; i++)
    {
        printf("\nEnter details for P%d\n", i);

        printf("Enter allocation -- ");
        for (int j = 0; j < m; j++)
        {
            scanf("%d", &alloc[i][j]);
        }

        printf("Enter Max -- ");
        for (int j = 0; j < m; j++)
        {
            scanf("%d", &max[i][j]);
        }
    }
    printf("\nEnter Available Resources -- ");
    for (int i = 0; i < m; i++)
    {
        scanf("%d", &avail[i]);
    }
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
```



```

    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int pid;
int request[m];

printf("\nEnter New Request Details --\n");

printf("Enter pid -- ");
scanf("%d", &pid);

printf("Enter Request for Resources -- ");
for (int i = 0; i < m; i++)
{
    scanf("%d", &request[i]);
}

int possible = 1;

for (int i = 0; i < m; i++)
{
    if (request[i] > need[pid][i] || request[i] > avail[i])
    {
        possible = 0;
        break;
    }
}

if (!possible)
{
    printf("\nRequest cannot be granted.\n");
    return 0;
}

for (int i = 0; i < m; i++)
{
    avail[i] -= request[i];
    alloc[pid][i] += request[i];
    need[pid][i] -= request[i];
}

for (int i = 0; i < n; i++)

```

```

{
    finish[i] = 0;
}

int count = 0;

while (count < n)
{
    int found = 0;

    for (int i = 0; i < n; i++)
    {
        if (finish[i] == 0)
        {
            int flag = 1;

            for (int j = 0; j < m; j++)
            {
                if (need[i][j] > avail[j])
                {
                    flag = 0;
                    break;
                }
            }

            if (flag)
            {
                for (int j = 0; j < m; j++)
                {
                    avail[j] += alloc[i][j];
                }

                printf("\nP%d is visited(", i);

                for (int j = 0; j < m; j++)
                {
                    printf("%d", avail[j]);

                    if (j != m - 1)
                        printf(" ");
                }

                printf(")");
            }
        }
    }

    count++;
}

```

```

        safe[count] = i;
        count++;

        finish[i] = 1;
        found = 1;
    }
}

if (found == 0)
{
    printf("\nSYSTEM IS NOT SAFE STATE\n");
    return 0;
}

printf("\n\nSYSTEM IS IN SAFE
STATE\n"); printf("The Safe Sequence is --

");

for (int i = 0; i < n; i++)
{
    printf("P%d", safe[i]);

    if (i != n - 1)
        printf(" ");
}

printf("\n\n");

return 0;
}

```

OUTPUT:

```
1BF24CS319
Enter number of processes -- 5
Enter number of resources -- 3

Enter details for P0
Enter allocation -- 0 1 0
Enter Max -- 7 5 3

Enter details for P1
Enter allocation -- 2 0 0
Enter Max -- 3 2 2

Enter details for P2
Enter allocation -- 3 0 2
Enter Max -- 9 0 2

Enter details for P3
Enter allocation -- 2 1 1
Enter Max -- 4 3 3

Enter details for P4
Enter allocation -- 0 0 2
Enter Max -- 4 3 3

Enter Available Resources -- 3 3 2

Enter New Request Details --
Enter pid -- 1
Enter Request for Resources -- 1 0 2
```

```
Enter allocation -- 2 0 0
Enter Max -- 3 2 2

Enter details for P2
Enter allocation -- 3 0 2
Enter Max -- 9 0 2

Enter details for P3
Enter allocation -- 2 1 1
Enter Max -- 4 3 3

Enter details for P4
Enter allocation -- 0 0 2
Enter Max -- 4 3 3

Enter Available Resources -- 3 3 2

Enter New Request Details --
Enter pid -- 1
Enter Request for Resources -- 1 0 2

P1 is visited(5 3 2)
P3 is visited(7 4 3)
P4 is visited(7 4 5)
P0 is visited(7 5 5)
P2 is visited(10 5 7)

SYSTEM IS IN SAFE STATE
The Safe Sequence is -- (P1 P3 P4 P0 P2)
thejesh@192:~/Documents/Bankers$
```

b) Deadlock

Detection #include

<stdio.h>

```
int main()
{
    int n, m;

    printf("Enter no of processes: ");
    scanf("%d", &n);

    printf("Enter no of resources: ");
    scanf("%d", &m);

    int alloc[n][m], req[n][m], avail[m], work[m];
    int finish[n], seq[n];

    for (int i = 0; i < n; i++)
    {
        printf("\nP%d Allocation: ", i);
        for (int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

        printf("P%d Request: ", i);
        for (int j = 0; j < m; j++)
            scanf("%d", &req[i][j]);
    }

    printf("\nAvailable Resources: ");
    for (int i = 0; i < m; i++)
    {
        scanf("%d", &avail[i]); work[i] = avail[i];
    }

    for (int i = 0; i < n; i++)
    {
        int sum = 0;
        finish[i] = 0;

        for (int j = 0; j < m; j++)
            sum += alloc[i][j];
```

```
if (sum == 0)
```

```

        finish[i] = 1;
    }

    int k = 0, found;

    do
    {
        found = 0;

        for (int i = 0; i < n; i++)
        {
            if (!finish[i])
            {
                int flag = 1;

                for (int j = 0; j < m; j++)
                {
                    if (req[i][j] > work[j])
                    {
                        flag = 0;
                        break;
                    }
                }

                if (flag)
                {
                    for (int j = 0; j < m; j++)
                        work[j] += alloc[i][j];

                    finish[i] = 1;
                    seq[k++] =
                    i; found = 1;

                    printf("P%d visited\n", i);
                }
            }
        }
    } while (found);

    int deadlock = 0;

    for (int i = 0; i < n; i++)
    {

```

```

    if (!finish[i])
    {
        deadlock = 1;
        break;
    }
}

if (deadlock)
{
    printf("\nSystem is in Deadlock\n");

    printf("Deadlocked Processes: ");
    for (int i = 0; i < n; i++)
        if (!finish[i])
            printf("P%d ", i);
}
else
{
    printf("\nSystem is not in Deadlock\n");

    printf("Sequence: ");
    for (int i = 0; i < k; i++)
        printf("P%d ", seq[i]);
}

return 0;
}

```

OUTPUT:


```
• thejesh@192:~/Documents/Deadlock$ /home
1BF24CS319
Enter no of processes: 5
Enter no of resources: 3

P0 Allocation: 0 1 0
P0 Request: 0 0 0

P1 Allocation: 2 0 0
P1 Request: 2 0 2

P2 Allocation: 3 0 3
P2 Request: 0 0 1

P3 Allocation: 2 1 1
P3 Request: 1 0 0

P4 Allocation: 0 0 2
P4 Request: 0 0 2

Available Resources: 0 0 0
P0 visited

System is in Deadlock
```

6. Write a C program to simulate the following contiguous memory allocation techniques.
(Any one) a) Worst-fit b) Best-fit c) First-fit

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 20

// Function prototypes with clear variable names
void firstFitFixed(int blocks[], int numBlocks, int processes[], int numProcesses);
void bestFitFixed(int blocks[], int numBlocks, int processes[], int numProcesses);
void worstFitFixed(int blocks[], int numBlocks, int processes[], int numProcesses);
void printAllocation(int processes[], int numProcesses, int allocation[]);

int main() {
    // Renamed m -> numBlocks and n -> numProcesses
    int numBlocks = 5;
    int numProcesses = 4;

    int blocks[] = {100, 500, 200, 300, 600};
    int processes[] = {212, 417, 112, 426};

    printf("--- Fixed Partition Allocation (Clear Variables) ---\n");

    firstFitFixed(blocks, numBlocks, processes, numProcesses);
    bestFitFixed(blocks, numBlocks, processes, numProcesses);
    worstFitFixed(blocks, numBlocks, processes, numProcesses);

    return 0;
}

// 1. FIRST-FIT
void firstFitFixed(int blocks[], int numBlocks, int processes[], int numProcesses) {
    int allocation[MAX];
    bool blockOccupied[MAX];

    for (int i = 0; i < numProcesses; i++) allocation[i] = -1;
    for (int j = 0; j < numBlocks; j++) blockOccupied[j] = false;

    for (int i = 0; i < numProcesses; i++) {
        for (int j = 0; j < numBlocks; j++) {
            if (!blockOccupied[j] && blocks[j] >= processes[i]) {
                allocation[i] = j;
                blockOccupied[j] = true;
                break;
            }
        }
    }
}

printf("\n--> FIRST-FIT RESULTS <--\n");
```

```

    printAllocation(processes, numProcesses, allocation);
}

// 2. BEST-FIT
void bestFitFixed(int blocks[], int numBlocks, int processes[], int numProcesses) {
    int allocation[MAX];
    bool blockOccupied[MAX];

    for (int i = 0; i < numProcesses; i++) allocation[i] = -1;
    for (int j = 0; j < numBlocks; j++) blockOccupied[j] = false;

    for (int i = 0; i < numProcesses; i++) {
        int bestIdx = -1;
        for (int j = 0; j < numBlocks; j++) {
            if (!blockOccupied[j] && blocks[j] >= processes[i]) {
                if (bestIdx == -1 || blocks[j] < blocks[bestIdx]) {
                    bestIdx = j;
                }
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blockOccupied[bestIdx] = true;
        }
    }
    printf("\n--> BEST-FIT RESULTS <--\n");
    printAllocation(processes, numProcesses, allocation);
}

// 3. WORST-FIT
void worstFitFixed(int blocks[], int numBlocks, int processes[], int numProcesses) {
    int allocation[MAX];
    bool blockOccupied[MAX];

    for (int i = 0; i < numProcesses; i++) allocation[i] = -1;
    for (int j = 0; j < numBlocks; j++) blockOccupied[j] = false;

    for (int i = 0; i < numProcesses; i++) {
        int worstIdx = -1;
        for (int j = 0; j < numBlocks; j++) {
            if (!blockOccupied[j] && blocks[j] >= processes[i]) {
                if (worstIdx == -1 || blocks[j] > blocks[worstIdx]) {
                    worstIdx = j;
                }
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blockOccupied[worstIdx] = true;
        }
    }
}

```

```

    }
}
printf("\n--> WORST-FIT RESULTS <--\n");
printAllocation(processes, numProcesses, allocation);
}

void printAllocation(int processes[], int numProcesses, int allocation[]) {
    printf("-----\n");
    printf("| Process No. | Process Size | Block No. Allocated |\n");
    printf("-----\n");
    for (int i = 0; i < numProcesses; i++) {
        printf("| %-11d | %-12d | ", i + 1, processes[i]);
        if (allocation[i] != -1) {
            printf("| %-19d |\n", allocation[i] + 1);
        } else {
            printf("| %-19s |\n", "Not Allocated");
        }
    }
    printf("-----\n");
}

```

OUTPUT:

```

1BF24C5319
--- Fixed Partition Allocation (Clear Variables) ---

--> FIRST-FIT RESULTS <--
-----
| Process No. | Process Size | Block No. Allocated |
-----
| 1           | 212         | 2                   |
| 2           | 417         | 5                   |
| 3           | 112         | 3                   |
| 4           | 426         | Not Allocated      |
-----

--> BEST-FIT RESULTS <--
-----
| Process No. | Process Size | Block No. Allocated |
-----
| 1           | 212         | 4                   |
| 2           | 417         | 2                   |
| 3           | 112         | 3                   |
| 4           | 426         | 5                   |
-----

--> WORST-FIT RESULTS <--

```

```

--> WORST-FIT RESULTS <--
-----
| Process No. | Process Size | Block No. Allocated |
-----
| 1           | 212         | 5                   |
| 2           | 417         | 2                   |
| 3           | 112         | 4                   |
| 4           | 426         | Not Allocated      |
-----

```

7. Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal

```
#include <stdio.h>
```

```
int search(int key, int frame[], int f) {  
    for (int i = 0; i < f; i++) {  
        if (frame[i] == key)  
            return 1;  
    }  
    return 0;  
}
```

```
void fifo(int pages[], int n, int f)
```

```
{ int frame[f];  
  int index = 0, faults = 0;
```

```
  for (int i = 0; i < f; i++)  
      frame[i] = -1;
```

```
  printf("\nFIFO Page Replacement Process:\n");
```

```
  for (int i = 0; i < n; i++) {
```

```
      if (!search(pages[i], frame, f)) {
```

```
          frame[index] =  
          pages[i]; index = (index  
          + 1) % f; faults++;
```

```
          printf("PF    No.    %d:    ",
```

```
          faults); for (int j = 0; j < f; j+
```

```
          +) {  
              if (frame[j] != -1)  
                  printf("%d ",  
                  frame[j]); else  
                  printf("- ");  
          }  
          printf("\n");
```

```
      }  
  }  
  
  printf("FIFO Page Faults: %d\n", faults);  
}
```

```

void lru(int pages[], int n, int f) {
    int frame[f], time[f];
    int faults = 0, count = 0;

    for (int i = 0; i < f; i++)
        { frame[i] = -1;
          time[i] = 0;
        }

    printf("\nLRU Page Replacement Process:\n");

    for (int i = 0; i < n; i++) {

        int found = 0;

        for (int j = 0; j < f; j++) {
            if (frame[j] == pages[i]) {
                count++;
                time[j] =
                count; found =
                1; break;
            }
        }

        if (!found)

            { int pos =

              0;

              for (int j = 1; j < f; j++)
                  { if (time[j] <
                    time[pos])
                      pos = j;
                  }

              frame[pos] = pages[i];
              count++;
              time[pos] = count; faults+
              +;

              printf("PF No. %d: ", faults);

              for (int j = 0; j < f; j++) {
                  if (frame[j] != -1)

```

```

        printf("%d ",
        frame[j]); elseprintf("- ");
    }
    printf("\n");
}
}

printf("LRU Page Faults: %d\n", faults);
}

void optimal(int pages[], int n, int f)
{ int frame[f];
int faults = 0;

for (int i = 0; i < f; i++)
    frame[i] = -1;

printf("\nOptimal Page Replacement Process:\n");

for (int i = 0; i < n; i++) {

    if (!search(pages[i], frame, f))

        { int pos = -1, farthest = i + 1;

        for (int j = 0; j < f; j++)
            { if (frame[j] == -1) {
                pos = j;
                break;
            }
        }

        if (pos == -1) {

            int max = -1;

            for (int j = 0; j < f; j++) {

                int k;

                for (k = i + 1; k < n; k++)
                    { if (frame[j] ==
                    pages[k])
                        break;
                    }if (k > max) { max = k; pos = j;
                }
            }
        }
    }
}

```

```

        }
    }

    frame[pos] = pages[i];
    faults++;

    printf("PF No. %d: ", faults);

    for (int j = 0; j < f; j++) {
        if (frame[j] != -1)
            printf("%d ",
                frame[j]); else
            printf("- ");
        }
    printf("\n");
}

printf("Optimal Page Faults: %d\n", faults);
}

int main()

{ int f, n;

    printf("Enter the number of Frames: ");
    scanf("%d", &f);

    printf("Enter the length of reference string: ");
    scanf("%d", &n);

    int pages[n];

    printf("Enter the reference string: ");

    for (int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    fifo(pages, n, f);
    lru(pages, n, f);

```



```
    optimal(pages, n, f);  
  
    return 0;  
}
```

OUTPUT:

```
1BF24CS319  
Enter the number of Frames: 3  
Enter the length of reference string: 6  
Enter the reference string: 1 4 2 3 2 1  
  
FIFO Page Replacement Process:  
PF No. 1: 1 - -  
PF No. 2: 1 4 -  
PF No. 3: 1 4 2  
PF No. 4: 3 4 2  
PF No. 5: 3 1 2  
FIFO Page Faults: 5  
  
LRU Page Replacement Process:  
PF No. 1: 1 - -  
PF No. 2: 1 4 -  
PF No. 3: 1 4 2  
PF No. 4: 3 4 2  
PF No. 5: 3 1 2  
LRU Page Faults: 5  
  
Optimal Page Replacement Process:  
PF No. 1: 1 - -  
PF No. 2: 1 4 -  
PF No. 3: 1 4 2  
PF No. 4: 1 3 2  
Optimal Page Faults: 4
```